



Systemes paralleles et distribues

Projet : Parallel Ant Foraging

Javier andres TARAZONA JIMENEZ

Jose Daniel CHACON

Encadrant : JUVIGNY Xavier

École Nationale Supérieure des Techniques Avancées (ENSTA)
Majeure STIC
Mars 2026

Table des matières

1	Introduction	3
1.1	Objectifs de la simulation	3
2	Analyse du problème - Baseline	4
2.1	État de l'art	4
2.2	Profilage et identification des goulots d'étranglement	4
2.3	Résultats	5
2.3.1	Analyse des Résultats	6
3	Vectorisation	8
3.1	Stratégie de Vectorisation	8
3.1.1	Motivation	8
3.1.2	Du layout AoS au layout SoA	8
3.1.3	Maintien du baseline : un binaire, deux layouts	8
3.1.4	Choix d'implémentation : types compacts et seed par fourni	8
3.1.5	Application dans le noyau : <code>advance_time_soa</code>	9
3.1.6	Découplage de <code>SDL_Point</code> hors hot loop et adaptation du rendu	9
3.1.7	Validation et protocole de comparaison	9
3.2	Résultat du Profilage	9
3.3	Analyse des Résultats	10
4	Parallélisation en mémoire partagée (OpenMP)	12
4.1	Stratégie retenue	12
4.1.1	Objectif	12
4.1.2	Choix d'architecture	12
4.1.3	Boucles parallélisées	12
4.1.4	Correctness et absence de data races	12
4.1.5	Réduction sparse et robustesse d'ordre	13
4.1.6	Profilage OpenMP-safe	13
4.2	Protocole expérimental	13
4.3	Résultats	14
4.3.1	Calcul explicite du speedup mesuré	14
4.3.2	Indicateur de granularité m (cells touched)	16
4.3.3	Vérifications empiriques de correctitude	16
4.4	Analyse et discussion des résultats	17

4.4.1	Ce que l'on attendait globalement, et ce que l'on observe	17
4.4.2	Lecture Amdahl : borne théorique cohérente avec les mesures	17
4.4.3	K4 (évaporation) : attendu memory-bound, observé memory-bound	17
4.4.4	K1 (avance des fourmis) : attendu limité par la fusion, observé	17
4.4.5	Variabilité expérimentale et équilibrage de charge	18
4.4.6	Pourquoi la comparaison serial vs omp à 1 thread reste informative, mais limitée	18
4.4.7	Bilan	18

1 Introduction

1.1 Objectifs de la simulation

L'objectif principal de la simulation est de modéliser le problème de fourragement d'une colonie de fourmis artificielles sur un terrain 2D hétérogène, afin d'identifier des trajets efficaces entre la fourmilière et une source de nourriture. Le cadre retenu suit un modèle d'intelligence en essaim où des agents simples produisent un comportement collectif émergent à partir de règles locales.

Le modèle vise à reproduire une coordination indirecte entre agents via deux types de phéromones : un phéromone d'exploration vers la nourriture (V_1) et un phéromone de retour vers le nid (V_2). Chaque fourmi évolue avec deux états (*unloaded*/*loaded*) et applique une politique exploration–exploitation pilotée par le paramètre ε , avec des transitions d'état lors de l'arrivée sur la nourriture ou sur la fourmilière.

L'environnement a pour objectif de représenter une difficulté de déplacement réaliste : chaque cellule de la carte possède un coût de traversée dans $[0, 1]$, et les zones non franchissables sont traitées comme obstacles. Le paysage est généré de manière fractale afin d'introduire des gradients de difficulté non uniformes et de contraindre la dynamique des déplacements.

Enfin, la simulation poursuit un objectif d'évaluation fonctionnelle au niveau de la colonie : la quantité de nourriture rapportée au nid sert d'indicateur global de performance. Ce cadre constitue la base expérimentale du projet, avant les étapes d'optimisation algorithmique (vectorisation et parallélisation) décrites dans le contexte général.

2 Analyse du problème - Baseline

2.1 État de l'art

L'implémentation actuelle initialise d'abord l'environnement dans `fractal_land.cpp` en construisant une carte fractale par raffinement récursif de sous-grilles. Dans `ant_simu.cpp`, cette carte est ensuite normalisée pour ramener les altitudes dans l'intervalle $[0, 1]$, ce qui fixe le coût de déplacement utilisé par les fourmis pendant la simulation.

La population est ensuite instanciée avec un nombre fixe de fourmis, des positions initiales pseudo-aléatoires et des paramètres globaux du modèle (*exploration*, *bruit*, *évaporation*). La structure `pheromone` (dans `pheromone.hpp`) maintient deux valeurs de phéromones par cellule, un tampon de mise à jour, ainsi que des bordures fantômes marquées à -1 pour empêcher les traversées hors domaine.

À chaque itération, la boucle principale de `ant_simu.cpp` traite les événements SDL puis exécute `advance_time`. Cette routine appelle `ant::advance` pour chaque agent, applique l'évaporation globale des phéromones (`do_evaporation`) puis valide l'état suivant de la carte (`update`). L'évolution individuelle d'une fourmi, définie dans `ant.cpp`, combine un choix local de voisin (aléatoire ou guidé par le maximum de phéromone), l'accumulation d'un budget de mouvement (`consumed_time < 1`), le marquage local des phéromones et la mise à jour de l'état chargé/non chargé.

Le rendu de l'état simulé est assuré par `render.cpp` et `window.cpp`. Le rendu affiche le terrain, la position des fourmis, la distribution des deux phéromones et la courbe de nourriture cumulée. La couche fenêtre/render SDL est construite avec mécanismes de repli (*fallback*) pour conserver l'exécution même lorsque certaines options matérielles ne sont pas disponibles.

2.2 Profilage et identification des goulots d'étranglement

Le profilage a pour objectif d'isoler les coûts dominants de la simulation avant toute optimisation, en particulier avant les étapes de vectorisation et de parallélisation. Nous présentons ici la définition des blocs à mesurer, instrumentation temporelle, protocole d'exécution reproductible et structure des résultats attendus. Cette section décrit donc la méthode de profilage et les objets techniques mobilisés, sans présenter de résultats chiffrés.

TABLE 1 – Découpage des parties mesurées pour le profilage de la simulation

ID	Nature	Rôle	Périmètre
P0	Initialisation	Génération fractale du terrain	One-shot (démarrage)
P1	Initialisation	Normalisation des coûts du terrain	One-shot (démarrage)
P2	Initialisation	Création et placement de la population	One-shot (démarrage)
K0	Noyau simulation	Coût total du noyau par itération	Itératif
K1	Noyau simulation	Avance globale de toutes les fourmis	Itératif
K2	Noyau simulation	Logique interne hors marquage phéromones	Itératif
K3	Noyau simulation	Marquage local des phéromones	Itératif
K4	Noyau simulation	Évaporation globale des phéromones	Itératif
K5	Noyau simulation	Validation/swap de la carte de phéromones	Itératif
R0	Rendu SDL	Affichage (terrain, fourmis, phéromones, courbe)	Itératif, hors noyau
E0	Événements SDL	Traitement de <code>SDL_PollEvent</code>	Itératif, hors noyau

Les relations structurelles utilisées pour l’analyse des goulots sont explicites :

$$K0 = K1 + K4 + K5$$

$$K1 = K2 + K3 + \text{reste}(K1)$$

Les blocs $P0$, $P1$ et $P2$ sont des coûts *one-shot* d’initialisation et ne doivent pas être mélangés aux métriques par itération. Les blocs $R0$ (rendu SDL) et $E0$ (gestion des événements SDL) sont mesurés séparément et traités comme non parallélisables dans l’analyse du noyau de calcul.

La stratégie instrumentale côté C++ s’appuie sur `timing_profile.hpp` et `timing_profile.cpp`. Le fichier `timing_profile.cpp` instancie l’état global de profilage (`g_timing_profile_totals`, `g_timing_profile_enabled`), tandis que `timing_profile.hpp` fournit l’horloge monotone (`profile_now_ns`) et les accumulateurs fins (`profile_add_k2`, `profile_add_k3`). L’accumulation des durées est effectuée bloc par bloc dans la boucle benchmark du simulateur, ce qui permet de séparer coûts d’initialisation, coûts noyau et coûts hors noyau.

L’automatisation est assurée par `scripts/measure_temps.sh`. Le pipeline suit une séquence reproductible : compilation release, puis 5 répétitions d’exécution avec 1200 itérations par répétition, 200 itérations de *warm-up* exclues et 1000 itérations effectivement mesurées, en configuration `OMP_NUM_THREADS=1`. Le script consolide les artefacts méthodologiques (`raw_measurements.csv`, `init_times.csv`, `table_a.csv`, `table_b.csv`, `amdahl_prediction.csv`, `summary.md`) afin de préparer la lecture des goulots. Les artefacts générés sont :

- `raw_measurements.csv` : mesures brutes par répétition (compteurs en ns et valeurs dérivées par itération).
- `init_times.csv` : synthèse des coûts *one-shot* d’initialisation ($P0$, $P1$, $P2$).
- `table_a.csv` : métriques des blocs de profilage (moyenne, dispersion et part relative du noyau).
- `table_b.csv` : indicateurs globaux par répétition (T_{iter} , T_{kernel} , T_{render} , débit).
- `amdahl_prediction.csv` : valeurs théoriques de speedup issues du modèle d’Amdahl à partir de la fraction parallélisable estimée.
- `summary.md` : synthèse lisible du protocole et des tableaux générés.

La méthode de lecture priorise les métriques `ms/iter` et `% noyau` pour identifier les *hotspots* dominants. Les conclusions doivent d’abord cibler les blocs du noyau ($K0$ – $K5$), puis isoler l’impact des composants hors noyau ($R0$, $E0$) pour éviter les biais d’interprétation.

2.3 Résultats

Les résultats présentés ci-dessous proviennent de la campagne de mesure :

`results/test_time_measurements/20260225_111731/summary.md`. La configuration utilisée est la suivante : 5 répétitions, 1200 itérations par répétition, 200 itérations de *warm-up*, 1000 itérations mesurées, et exécution avec `OMP_NUM_THREADS=1`.

TABLE 2 – Temps one-shot d’initialisation

Partie	ms moyen	écart-type
P0	41.581492	0.852773
P1	0.891350	0.031879
P2	0.076836	0.010131

TABLE 3 – Décomposition des coûts (profilage de base)

Partie	ms/iter moyen	écart-type	% kernel
K1	3.719856	0.005782	93.4676
K2	2.812386	0.005215	70.6659
K3	0.580826	0.002620	14.5942
K4	0.255192	0.004034	6.4121
K5	0.004597	0.000359	0.1155
R0	0.948657	0.011420	n/a
E0	0.001842	0.000113	n/a

TABLE 4 – Indicateurs globaux par répétition

Répétition	T_iter_total (ms)	T_kernel (ms)	T_render (ms)	fourmis/s
1	4.951205	3.989457	0.959952	1253303.431806
2	4.951423	3.991688	0.957977	1252602.984471
3	4.915457	3.972775	0.940620	1258566.168530
4	4.934418	3.978167	0.954489	1256860.215796
5	4.899159	3.967081	0.930245	1260372.560533
Moyenne	4.930332	3.979834	0.948657	1256341.072227
Écart-type	0.020446	0.009470	0.011420	2989.114282

Fraction parallélisable candidate :

$$F = (K1 + K4)/K0 = 0.998797$$

TABLE 5 – Prédiction théorique selon Amdahl

Threads	S(p)
2	1.997597
4	3.985616
8	7.933195
16	15.716398

2.3.1 Analyse des Résultats

Cette sous-section interprète les mesures présentées dans les tableaux 2, 3, 4 et 5, ainsi que les indicateurs F et $S(p)$. L’objectif est d’identifier les priorités de parallélisation à partir des données observées, en séparant clairement ce qui relève des coûts d’initialisation, du noyau de simulation et des bornes théoriques de gain.

1. **Coûts d’initialisation ($P0$ – $P2$).** Dans le tableau 2, $P0$ domine très largement la phase d’initialisation (41.581492 ms), alors que $P1$ (0.891350 ms) et $P2$ (0.076836 ms) restent faibles. Cette hiérarchie est cohérente avec le fait que la génération fractale du terrain est la tâche la plus lourde au démarrage. En revanche, comme $P0$ – $P2$ sont des coûts *one-shot* hors boucle itérative, ils ne constituent pas la priorité principale pour améliorer le temps de simulation par itération.
2. **Priorité de parallélisation au niveau noyau.** D’après le tableau 3, $K1$ représente 93.4676% du temps noyau, ce qui en fait la cible prioritaire. Le raisonnement est direct : réduire $K1$ a

l'impact potentiel maximal sur $K0$, avant $K4$ (évaporation) et $K5$ (mise à jour de la carte de phéromones), dont les contributions relatives sont nettement plus faibles.

3. **Décomposition interne de $K1$: $K2$ avant $K3$.** Toujours dans le tableau 3, $K2$ pèse 70.6659% du noyau contre 14.5942% pour $K3$. L'ordre de priorité est donc de paralléliser d'abord $K2$, puis $K3$. Cela confirme que la logique de déplacement/décision des fourmis est plus coûteuse que le marquage des phéromones.
4. **Stabilité expérimentale inter-répétitions.** Le tableau 4 montre une faible dispersion entre les 5 répétitions ($T_{\text{iter_total}}$, T_{kernel} , T_{render} , fourmis/s), ce qui indique une bonne stabilité de la campagne de mesure. Cette stabilité provient du protocole (phase de *warm-up* exclue et répétitions multiples), et non d'un effet spécifique "à partir de la cinquième itération".
5. **Interprétation de la fraction parallélisable F .** La valeur $F = 0.998797$, proche de 1, n'est pas anormale dans ce cadre : la définition $F = (K1 + K4)/K0$ inclut les blocs dominants du noyau et exclut les parties hors noyau ($R0$, $E0$). Cette valeur doit toutefois rester interprétée comme une fraction candidate théorique pour le noyau, et non comme une garantie de speedup réel, car les surcoûts de synchronisation, d'accès mémoire et d'équilibrage peuvent réduire le gain observé.
6. **Lecture correcte de la prédiction d'Amdahl.** Le tableau 5 montre un potentiel de gain important avec l'augmentation du nombre de threads. En particulier, $S(16) = 15.716398$ correspond à un facteur multiplicatif d'environ $\times 15.7$, et non à 15%. Cette tendance soutient clairement la stratégie de parallélisation ; la suite du projet doit donc viser d'abord $K1$, puis $K4$, pour se rapprocher de ce plafond théorique.

En synthèse, les deux *hotspots* centraux sont $K1$ au niveau global et $K2$ à l'intérieur de $K1$. Les mesures actuelles établissent une base robuste pour prioriser les optimisations, mais l'écart entre potentiel théorique et performance finale devra être vérifié expérimentalement après implémentation de la parallélisation. La feuille de route est donc cohérente : concentrer l'effort d'abord sur les blocs dominants du noyau afin de maximiser le retour de chaque évolution de code.

3 Vectorisation

3.1 Stratégie de Vectorisation

3.1.1 Motivation

Le point 5.1 de l'énoncé définit la *vectorisation* comme un changement de représentation des agents : au lieu de manipuler une liste d'objets « fourmi », on stocke séparément les attributs (coordonnées, état, graine de générateur pseudo-aléatoire, etc.) dans des tableaux. Une fourmi est alors repérée uniquement par son indice i dans ces tableaux.

Cette étape est motivée par l'analyse *baseline* : le bloc $K1$ (avance de l'ensemble des fourmis) concentre l'essentiel du temps noyau. L'objectif principal est donc d'améliorer l'efficacité mémoire de cette boucle (localité, cache, bande passante) afin de réduire le coût par itération, tout en préparant la suite (parallélisation OpenMP au point 5.2 de la consigne).

3.1.2 Du layout AoS au layout SoA

Dans la version *baseline*, la population est stockée sous la forme d'un `std::vector<ant>` (*Array of Structs*, AoS). Chaque instance `ant` regroupe plusieurs champs hétérogènes (position `SDL_Point`, état, graine RNG, etc.). Lors de l'itération sur toutes les fourmis, le noyau lit/écrit principalement la position, l'état et la graine, mais les lignes de cache chargées contiennent aussi des données non nécessaires au pas courant.

Nous introduisons donc un layout *Structure of Arrays* (SoA), plus favorable à la localité et à la vectorisation au sens « données » du sujet :

$$\text{AoS} : \{(x_i, y_i, s_i, g_i)\}_{i=0}^{N-1} \longrightarrow \text{SoA} : (x[], y[], s[], g[]).$$

Ainsi, les accès séquentiels `x[i]`, `y[i]`, `state[i]`, `seed[i]` deviennent contigus en mémoire, ce qui augmente la proportion de données utiles par ligne de cache et facilite les boucles par indice.

3.1.3 Maintien du baseline : un binaire, deux layouts

Afin de conserver une référence expérimentale stable, le chemin AoS est maintenu et reste le mode d'exécution par défaut. Le programme expose un paramètre de ligne de commande permettant de choisir la représentation :

- `-layout aos` : exécution *baseline* (`std::vector<ant>`);
- `-layout soa` : exécution vectorisée (`AntsSoA`).

Ce choix « 1 binaire, 2 layouts » évite de dupliquer le projet en deux répertoires (et donc de diverger au fil des modifications) et garantit une comparaison directe AoS vs SoA à paramètres identiques.

3.1.4 Choix d'implémentation : types compacts et seed par fourmi

En SoA, nous stockons explicitement les attributs minimaux requis par la dynamique :

- `x[i]`, `y[i]` en `int32_t` (coordonnées);
- `state[i]` en `uint8_t` (0 = *unloaded*, 1 = *loaded*);
- `seed[i]` en `uint32_t` (état RNG local).

Le choix de types compacts réduit l’empreinte mémoire par agent et améliore l’utilisation du cache, ce qui est particulièrement pertinent puisque la boucle $K1$ parcourt plusieurs milliers de fourmis.

Un point important pour la reproductibilité est la gestion de la graine RNG par fourmi. Dans le code AoS *baseline*, le champ `m_seed` est utilisé dans `ant::advance (src/ant.cpp)` mais n’était pas initialisé dans le constructeur (`src/ant.hpp`), ce qui induit un *comportement indéfini* et rend le baseline instable. La vectorisation impose de rendre cette donnée explicite : chaque fourmi possède sa graine `seed[i]` et les fonctions de génération aléatoire la mettent à jour par référence. Pour garder une comparabilité maximale, la formule du générateur est conservée ; seule l’initialisation devient correcte et déterministe.

3.1.5 Application dans le noyau : `advance_time_soa`

La version SoA introduit un noyau `advance_time_soa` (analogue à `advance_time` dans `src/ant_simu.cpp`) :

1. boucle sur $i \in [0, N)$: avance de la fourmi i en manipulant `x[i]`, `y[i]`, `state[i]`, `seed[i]` ;
2. évaporation globale des phéromones (`do_evaporation`) ;
3. validation/swap de la carte de phéromones (`update`).

La logique par fourmi reste volontairement identique à la version AoS (mêmes choix de voisin, mêmes branches, même condition de mouvement `consumed_time < 1`) afin de comparer les performances en isolant l’effet du layout mémoire.

3.1.6 Découplage de `SDL_Point` hors hot loop et adaptation du rendu

Le type `SDL_Point` est pratique pour le rendu mais il couple le noyau de calcul à SDL. Dans la stratégie SoA, le noyau manipule des scalaires `int32_t` ; la conversion vers un point SDL (si nécessaire) est repoussée au moment du rendu. Dans le même esprit, une surcharge `mark_pheronome(x, y)` côté `pheronome` permet de marquer les phéromones sans construire un `SDL_Point` à chaque pas.

Enfin, le `Renderer` est adapté pour accepter soit une population AoS (`std::vector<ant>`), soit une population SoA (`AntsSoA`). Le rendu itère alors :

- en AoS : sur les objets et `ant.get_position()` ;
- en SoA : sur l’indice i et les tableaux `x[i]/y[i]`.

3.1.7 Validation et protocole de comparaison

La validation repose sur la compilation en mode *release* puis l’exécution du mode `-benchmark` en forçant successivement `-layout aos` puis `-layout soa`. Le format des métriques `METRIC ...` est conservé afin de réutiliser le pipeline de consolidation existant (`scripts/measure_temps.sh`). La comparaison se focalise sur $K1$ (et donc $K0$), puisque la vectorisation vise prioritairement le noyau d’avance des fourmis.

3.2 Résultat du Profilage

Les résultats présentés ci-dessous proviennent de la campagne de mesure :

`results/test_vectorisation_layout/soa/20260227_115411/summary.md`. La configuration utilisée est la suivante : layout `soa`, 5 répétitions, 1200 itérations par répétition, 200 itérations de *warm-up*, 1000 itérations mesurées, et exécution avec `OMP_NUM_THREADS=1`.

TABLE 6 – Temps one-shot d’initialisation (layout SoA)

Partie	ms moyen	écart-type
P0	44.024725	1.211201
P1	0.951707	0.044184
P2	0.072038	0.007817

TABLE 7 – Décomposition des coûts (profilage, layout SoA)

Partie	ms/iter moyen	écart-type	% kernel
K1	4.264492	0.022742	94.0300
K2	3.230802	0.017821	71.2376
K3	0.668624	0.005969	14.7428
K4	0.266004	0.004142	5.8653
K5	0.004469	0.000404	0.0985
R0	1.024001	0.013386	n/a
E0	0.002005	0.000105	n/a

TABLE 8 – Indicateurs globaux par répétition (layout SoA)

Répétition	T_iter_total (ms)	T_kernel (ms)	T_render (ms)	fourmis/s
1	5.572491	4.550202	1.020098	1098852.270736
2	5.528406	4.507923	1.018436	1109158.323270
3	5.615466	4.574892	1.038652	1092921.937721
4	5.581273	4.540269	1.039038	1101256.219797
5	5.508628	4.502948	1.003779	1110383.627732
Moyenne	5.561253	4.535247	1.024001	1102514.475851
Écart-type	0.038248	0.026872	0.013386	6528.031897

Fraction parallélisable candidate (layout SoA) :

$$F = (K1 + K4)/K0 = 0.998952$$

TABLE 9 – Prédiction théorique selon Amdahl (layout SoA)

Threads	S(p)
2	1.997906
4	3.987463
8	7.941739
16	15.752373

3.3 Analyse des Résultats

Afin d’évaluer l’impact de la vectorisation SoA de manière contrôlée, nous comparons deux campagnes de mesure exécutées sur la même machine, avec les mêmes paramètres et le même script de profilage, à 43 s d’intervalle :

— AoS (baseline) : `results/test_vectorisation_layout/aos/20260227_115454/summary.md` ;

— SoA (vectorisé) : `results/test_vectorisation_layout/soa/20260227_115411/summary.md`.

La variation relative est définie par :

$$\Delta_{\%}(X) = 100 \times \frac{X_{\text{SoA}} - X_{\text{AoS}}}{X_{\text{AoS}}}.$$

TABLE 10 – Comparaison AoS vs SoA (moyennes sur 5 répétitions)

Métrique	AoS	SoA	Δ	$\Delta_{\%}$
T_{kernel} (ms/iter)	4.533697	4.535247	+0.001550	+0.034
$K1$ (ms/iter)	4.257748	4.264492	+0.006744	+0.158
$R0$ (ms/iter)	1.040353	1.024001	-0.016352	-1.572
$T_{\text{iter,total}}$ (ms/iter)	5.576019	5.561253	-0.014766	-0.265
fourmis/s	1102929.095441	1102514.475851	-414.619590	-0.038

1. **Pas de speedup mesurable sur le noyau (K0/K1).** Les temps noyau T_{kernel} ($\approx K0$) sont quasi identiques : AoS 4.533697 ± 0.037727 contre SoA 4.535247 ± 0.026872 ms/iter, soit $\Delta_{\%} = +0.034\%$ (légère dégradation SoA). De même, $K1$ passe de 4.257748 ± 0.030779 (AoS) à 4.264492 ± 0.022742 ms/iter (SoA), soit $\Delta_{\%} = +0.158\%$. Ces écarts sont très inférieurs aux dispersions inter-répétitions (écarts-types), ce qui ne permet pas de conclure à un gain significatif sur le noyau via le seul changement de layout des attributs des fourmis.
2. **La distribution des hotspots reste inchangée.** Dans les deux layouts, $K1$ représente environ 94% du temps noyau (AoS : 93.9134%, SoA : 94.0300%). Au sein de $K1$, $K2$ reste la composante dominante : il pèse environ 71% du noyau (AoS : 71.1255%, SoA : 71.2376%). Autrement dit, le coût principal demeure la logique interne d’avance des fourmis (branchements, choix aléatoires, condition `consumed_time < 1`, et surtout accès/écritures indirectes sur `pheromone` et accès à `land`), plutôt que le stockage des attributs `x/y/state/seed`.
3. **Légère amélioration du rendu, sans impact sur le calcul.** Le coût de rendu $R0$ diminue de 1.040353 ± 0.015469 (AoS) à 1.024001 ± 0.013386 ms/iter (SoA), soit $\Delta_{\%} = -1.572\%$. Cette baisse explique la petite amélioration du temps total par itération ($\Delta_{\%} = -0.265\%$), alors même que le noyau de calcul reste inchangé. Le gain observé est donc principalement hors noyau.
4. **Débit quasi identique.** Le débit passe de $1102929.095441 \pm 9173.468667$ (AoS) à $1102514.475851 \pm 6528.031897$ fourmis/s (SoA), soit $\Delta_{\%} = -0.038\%$. Cette stabilité est cohérente avec l’absence de gain sur T_{kernel} , puisque le débit (tel que calculé par le script) dépend essentiellement du temps noyau.
5. **Interprétation et suite.** La vectorisation SoA constitue surtout une *restructuration* conforme au point 5.1 (tableaux par attribut, fourni repérée par indice) et un prérequis pour la suite (5.2 OpenMP, et potentielles transformations plus profondes). En revanche, ces mesures suggèrent qu’une amélioration de performance du noyau exigera d’attaquer la partie dominante $K2$ (accès mémoire sur la carte de phéromones, organisation des mises à jour, réduction de divergence du `while`, parallélisation), plutôt que le seul layout des attributs agents.

4 Parallélisation en mémoire partagée (OpenMP)

4.1 Stratégie retenue

4.1.1 Objectif

Après la vectorisation (layout SoA), l'objectif du point OpenMP est de réduire le temps noyau $K0$ en parallélisant des boucles à fort coût, tout en conservant une sortie benchmark reproductible (METRIC ...) pour l'analyse de speedup.

4.1.2 Choix d'architecture

L'implémentation OpenMP est portée par le backend `omp+soa` (`src/sim/omp_soa_backend.cpp`) et sélectionnée via :

```
— -exec omp
— -layout soa
```

Le backend `omp+aos` a été volontairement retiré pour rester aligné avec le fil logique de la consigne (vectoriser, puis paralléliser la version vectorisée).

4.1.3 Boucles parallélisées

1. **Évaporation des phéromones ($K4$).** La fonction `do_evaporation()` (dans `src/pheronome.hpp`) applique `#pragma omp parallel for collapse(2) schedule(static)` sur le balayage de grille. Chaque cellule est indépendante pendant cette phase; il n'y a donc pas de conflit d'écriture.
2. **Avance des fourmis ($K1$) avec buffers locaux par thread.** Dans `OmpSoaBackend::step`, la boucle sur les indices de fourmis est parallélisée avec `#pragma omp for schedule(static)`. Pendant la région parallèle, un thread :
 - met à jour ses fourmis locales (`x[i]`, `y[i]`, `state[i]`, `seed[i]`),
 - enregistre les cellules touchées dans `touched_v1/touched_v2`,
 - accumule son `food_delta` local.

Aucune écriture globale de phéromone n'est faite à l'intérieur de la région parallèle.

4.1.4 Correctness et absence de data races

Les points sensibles en mémoire partagée sont :

- la carte de phéromones (écritures concurrentes potentielles),
- le compteur global `food_quantity` (incréments concurrents).

Le schéma retenu évite ces conflits :

- chaque thread écrit uniquement dans ses structures locales (`touched_v1/touched_v2`, `food_delta`),
- les deltas de nourriture sont fusionnés après la région parallèle,
- les marquages de phéromones sont rejoués séquentiellement sur la carte globale.

On évite ainsi les *write/write races* sur la grille et les *read/modify/write races* sur le compteur de nourriture.

4.1.5 Réduction sparse et robustesse d'ordre

Après la région parallèle :

1. concaténation des vecteurs `touched` de tous les threads,
2. tri + `unique` pour ne garder qu'une fois chaque cellule,
3. application séquentielle des marques sur la carte globale.

Ce schéma est cohérent avec la modification de `mark_pheronome` vers une mise à jour idempotente par itération (type MAX). Cela suit l'esprit de l'énoncé : l'ordre de traitement des fourmis peut être arbitraire en parallèle, et la politique de fusion par maximum sert à rendre le résultat robuste à cet ordre. Cette propriété est aussi utile pour les futures réductions globales (OpenMP/MPI).

4.1.6 Profilage OpenMP-safe

Pour éviter les courses sur les accumulateurs de profilage, les mesures fines dans les régions parallèles ont été retirées. Le profilage est conservé aux frontières de kernels ($K1$, $K4$, $K5$).

Plus précisément, en version serial, $K2$ et $K3$ étaient incrémentés à très haute fréquence dans la logique interne de chaque fourmi ($K2$: décision/déplacement ; $K3$: marquage). En OpenMP, cette instrumentation se trouve à l'intérieur de la région parallèle. Si on la conserve telle quelle, deux problèmes apparaissent :

- **problème de sûreté/concurrence** : plusieurs threads mettraient à jour les mêmes accumulateurs (race), ce qui imposerait des `atomic/critical` très fréquents ;
- **problème de validité de la mesure** : sommer des temps par thread ne donne pas un temps mur (*wall-time*) de $K1$; on peut obtenir des valeurs difficilement comparables au baseline (par exemple $\sum_t K2_t$ peut dépasser $K1$).

Ajouter des atomiques pour "remplir" $K2/K3$ introduirait en plus un surcoût et de la contention (voire du false sharing), ce qui perturberait la performance mesurée elle-même. Pour cette raison, en campagne OpenMP, on garde un profilage robuste aux frontières de phase ($K1$, $K4$, $K5$) et $K2/K3$ sont laissés à zéro par choix méthodologique.

4.2 Protocole expérimental

Campagnes utilisées pour cette section :

- OpenMP SoA : `results/test_openmp_report_v2/omp/soa/latest`
- Référence serial SoA (1 thread) : `results/test_openmp_report_v2/serial/soa/latest`

Configuration commune :

- 1200 itérations, 200 warm-up, donc 1000 itérations mesurées,
- mode `-benchmark -no-render`,
- 7 répétitions par configuration,
- machine : AMD Ryzen 7 5800H (8 coeurs physiques / 16 threads logiques), WSL2,
- sweep $\{1, 2, 4, 8\}$ aligné sur les coeurs physiques (pas de point 16 ici),
- variables d'affinité OpenMP `OMP_PROC_BIND/OMP_PLACES` non fixées dans cette campagne (valeurs par défaut runtime).

Commande type :

```
./scripts/measure_openmp.sh --folder test_openmp_report_v2 --layout soa --exec omp \
  --threads 1,2,4,8 --iterations 1200 --warmup 200 --reps 7
```

4.3 Résultats

TABLE 11 – OpenMP SoA : temps noyau et speedup selon le nombre de threads

Threads	K0 moyen (ms/iter)	écart-type	Speedup $S(p)$	Efficacité $E(p) = S/p$
1	4.333409	0.333818	1.000000	1.0000
2	3.600448	0.324506	1.203575	0.6018
4	3.455067	0.589087	1.254219	0.3136
8	2.802022	0.547734	1.546529	0.1933

4.3.1 Calcul explicite du speedup mesuré

Le speedup expérimental est défini par :

$$S(p) = \frac{T(1)}{T(p)}$$

où $T(p)$ est le temps moyen du noyau $K0$ (ms/itération) avec p threads. Dans cette section, la référence $T(1)$ est **le backend omp+soa à 1 thread** (et non **serial+soa**), afin de mesurer strictement l'effet du threading à algorithme constant.

À partir des valeurs mesurées :

$$T(1) = 4.333409, \quad T(2) = 3.600448, \quad T(4) = 3.455067, \quad T(8) = 2.802022$$

$$S(1) = \frac{4.333409}{4.333409} = 1.000000$$

$$S(2) = \frac{4.333409}{3.600448} = 1.203574944 \approx 1.203575$$

$$S(4) = \frac{4.333409}{3.455067} = 1.254218514 \approx 1.254219$$

$$S(8) = \frac{4.333409}{2.802022} = 1.546529256 \approx 1.546529$$

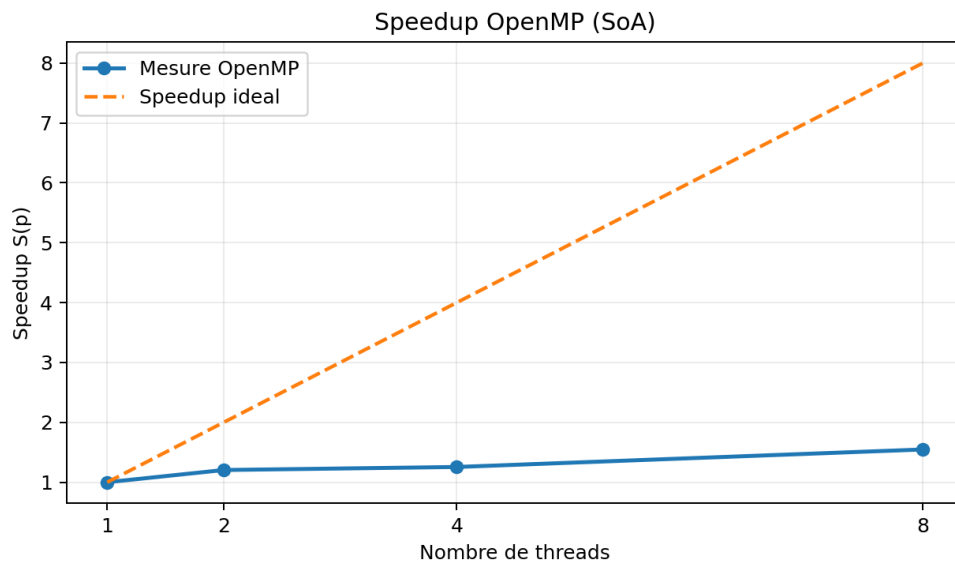


FIGURE 1 – Speedup mesuré OpenMP (SoA) vs speedup idéal linéaire

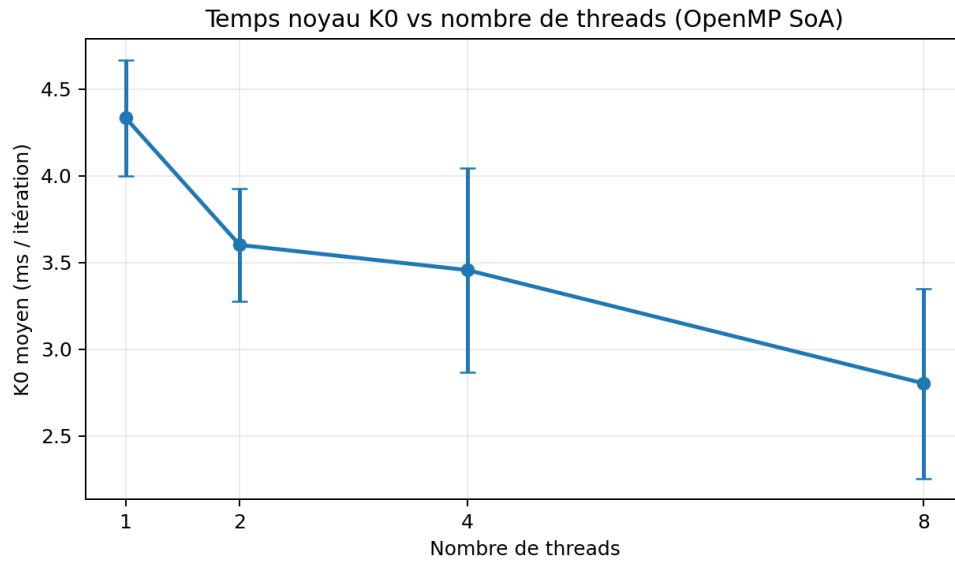
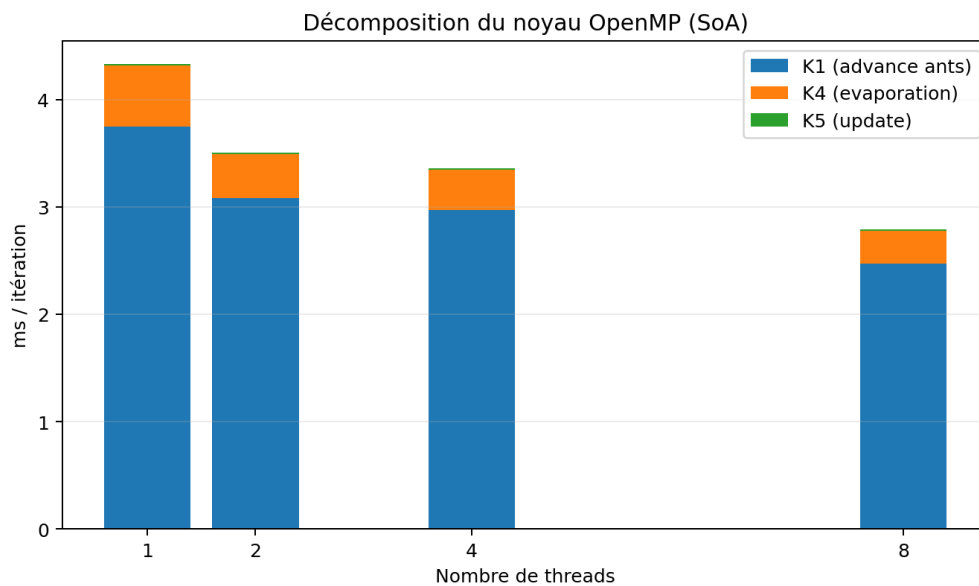
FIGURE 2 – Temps noyau $K0$ en fonction du nombre de threads (barres d'erreur : écart-type)

TABLE 12 – Décomposition du noyau OpenMP SoA

Threads	K1 (ms/iter)	K4 (ms/iter)	K5 (ms/iter)
1	3.746828	0.572646	0.010618
2	3.083960	0.410361	0.010715
4	2.970344	0.375694	0.013603
8	2.474847	0.303397	0.012672

FIGURE 3 – Décomposition $K1/K4/K5$ du noyau OpenMP SoA

4.3.2 Indicateur de granularité m (cells touched)

Pour quantifier le coût de post-traitement `sort+unique`, on suit :

$$m_{\text{raw}} = \# \text{cells touched (avant déduplication)} \quad \text{et} \quad m_{\text{uniq}} = \# \text{cells touched uniques}$$

ainsi que le ratio $m_{\text{uniq}}/m_{\text{raw}}$. Une campagne de contrôle (`iterations=5000`, `warmup=500`, `seed=2026`, `results/test_openmp_report_checks/latest`) donne :

TABLE 13 – Granularité observée pour la phase de fusion sparse

Threads	m_{raw} (cells/iter)	m_{uniq} (cells/iter)	$m_{\text{uniq}}/m_{\text{raw}}$
1	17992.3	14866.0	0.826245
8	17992.3	14866.0	0.826245

Ce niveau de m explique le coût non négligeable de la fusion : la complexité $\mathcal{O}(m \log m)$ de `sort+unique`, ajoutée au replay séquentiel, limite la scalabilité de *K1* même lorsque la boucle principale sur les fourmis est parallélisée.

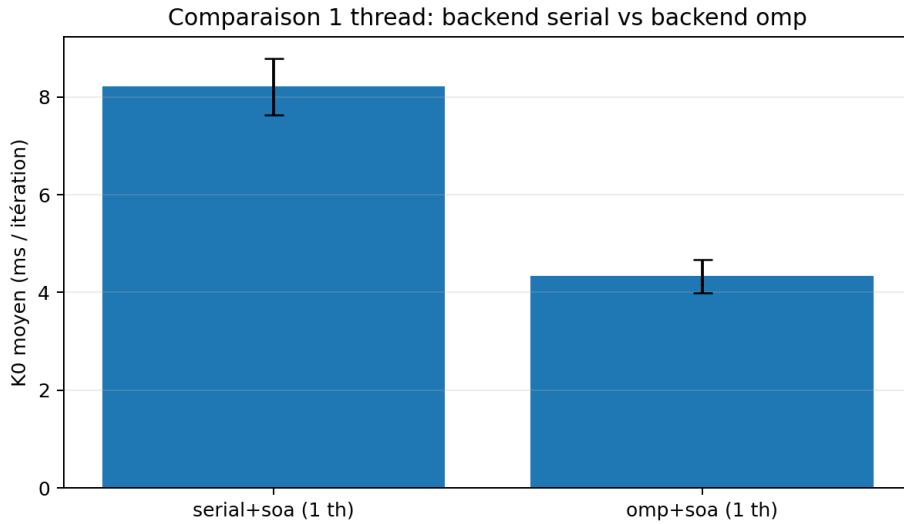


FIGURE 4 – Comparaison 1 thread : backend serial+soa vs backend omp+soa

4.3.3 Vérifications empiriques de correctitude

En complément de l’argument “absence de races”, on vérifie deux invariants simples :

- `food_quantity` monotone non décroissante (`food_monotonic_ok=1`),
- stabilité d’indicateurs globaux de phéromones entre 1 et 8 threads (même seed).

TABLE 14 – Checks de correctitude OpenMP (5000 itérations, warmup 500, seed 2026)

Métrique	OMP 1 thread	OMP 8 threads
food_monotonic_ok	1	1
food_quantity_final	1556	1556
phen_v1_sum	2989.08	2989.08
phen_v2_sum	16321.2	16321.2
phen_v1_max	1.0	1.0
phen_v2_max	1.0	1.0

4.4 Analyse et discussion des résultats

4.4.1 Ce que l'on attendait globalement, et ce que l'on observe

D'après la théorie, un code contenant des boucles largement parallélisables devrait accélérer avec le nombre de threads. Dans nos mesures, ce comportement est bien présent : $S(2) > S(1)$, $S(4) > S(2)$ et $S(8) > S(4)$. En revanche, l'accélération reste modérée :

$$S(8) = 1.5465 \quad (\text{vs idéal } 8.0)$$

Autrement dit, la parallélisation apporte un gain réel, mais on n'observe pas une scalabilité proche de la droite idéale.

4.4.2 Lecture Amdahl : borne théorique cohérente avec les mesures

Si l'on applique la loi d'Amdahl au point mesuré $S(8) = 1.546529$, on obtient :

$$f \approx \frac{8/S(8) - 1}{7} \approx 0.60, \quad S_{\max} \approx \frac{1}{f} \approx 1.68$$

Donc, compte tenu de la fraction effectivement séquentielle, il est logique d'obtenir un speedup qui plafonne autour de 1.6–1.7. Le résultat observé à 8 threads (1.55) est compatible avec cette borne.

4.4.3 K4 (évaporation) : attendu memory-bound, observé memory-bound

Le bloc $K4$ correspond à un balayage régulier de grille, indépendant cellule par cellule. On pouvait donc attendre une bonne parallélisation, mais non linéaire à cause de la bande passante mémoire. Les mesures confirment ce scénario : $K4$ passe de 0.572646 à 0.303397 ms/iter entre 1 et 8 threads ($\approx -47\%$), loin d'un facteur $\times 8$, ce qui est typique d'un régime limité par la mémoire/cache.

4.4.4 K1 (avance des fourmis) : attendu limité par la fusion, observé

On pouvait s'attendre à ce que $K1$ ne scale pas autant que $K4$, car la stratégie actuelle inclut une phase de fusion après la région parallèle. C'est exactement ce qu'on observe : $K1$ baisse de 3.746828 à 2.474847 ms/iter ($\approx -34\%$), soit moins que $K4$.

La raison est structurelle :

— phase parallèle : déplacement des fourmis + enregistrement des cellules touchées,

— phase séquentielle : concaténation, tri, **unique**, replay des marques.

Cette réorganisation ajoute un coût en $\mathcal{O}(m \log m)$ (avec m le nombre de cellules touchées), plus des accès mémoire irréguliers. On retrouve ici une limite classique de granularité : une partie du gain parallèle est reperdue dans le post-traitement séquentiel.

4.4.5 Variabilité expérimentale et équilibrage de charge

Les écarts-types augmentent avec le nombre de threads (Table 11), surtout à 4 et 8. C’est cohérent avec un environnement WSL2 (activité de fond, contention CPU/mémoire), et aussi avec la nature du calcul : même avec `schedule(static)`, le coût par fourmi n’est pas strictement constant (branches, durée du `while`, nombre de cellules touchées). Un déséquilibre de charge résiduel peut donc dégrader l’efficacité. Le choix `static` est conservé ici comme baseline à faible overhead ; une vérification complémentaire possible consiste à tester `schedule(dynamic, chunk)` sur *K1* pour comparer variance et throughput.

4.4.6 Pourquoi la comparaison serial vs omp à 1 thread reste informative, mais limitée

La Figure 4 montre un écart net entre `serial+soa` (1 thread) et `omp+soa` (1 thread). Cette comparaison indique bien que les deux backends n’ont pas exactement la même structure de coût (profilage interne, stratégie sparse dedup/replay), mais elle ne mesure pas à elle seule le gain dû au threading.

Pour quantifier l’effet du parallélisme OpenMP pur, la référence correcte reste $S(p) = T_1/T_p$ à l’intérieur du même backend OMP (Table 11).

4.4.7 Bilan

En résumé, les résultats sont cohérents avec ce qu’on pouvait attendre d’une implémentation OpenMP réaliste :

- accélération mesurable et reproductible ($S(8) \approx 1.55$),
- *K4* limité surtout par la mémoire,
- *K1* limité par la phase de fusion sparse (partie séquentielle).

La conclusion pratique est claire : pour dépasser le niveau actuel de speedup, il faut réduire la part séquentielle de la fusion post-parallèle.